

CV10: Dice Game - Teória

1 Úvod do cvičenia

Desiate cvičenie predstavuje záverečnú miniaplikáciu, v ktorej sa spájajú viaceré témy z predchádzajúcich cvičení do jedného menšieho celku. Používateľské rozhranie je vytvorené v Jetpack Compose, aplikácia obsahuje viac obrazoviek prepojených cez Navigation Compose, stav a logika sú spracované vo `ViewModel` triede a dáta sa ukladajú lokálne pomocou Room databázy.

Ako príklad sa použije aplikácia *Dice Game*. Používateľ má k dispozícii predvolenú kocku D6 a môže si pridávať vlastné kocky podľa počtu strán, napríklad D4, D20 alebo D100. Z hlavnej obrazovky si vyberie konkrétnu kocku, otvorí obrazovku hodu a môže zobrazíť aj štatistiku pre vybranú kocku.

2 Prehľad obrazoviek v aplikácii

Aplikácia obsahuje tri obrazovky. Prvá obrazovka slúži na výber kocky. Zobrazuje všetky dostupné kocky, umožňuje pridať novú a pri každej ponúka dve akcie. Používateľ môže otvoriť obrazovku hodu alebo obrazovku štatistiky.

Druhá obrazovka je určená pre samotný hod kockou. Po otvorení už pracuje iba s jednou konkrétnou kockou. Obsahuje tlačidlo na vykonanie hodu a zobrazuje posledný výsledok. Každý nový hod sa zároveň uloží do databázy.

Tretia obrazovka zobrazuje štatistiky pre konkrétnu kocku. Používateľ tu vidí počet hodov, posledný výsledok, najvyššiu a najnižšiu hodnotu, priemer a zoznam posledných hodov.

3 Navigácia medzi obrazovkami

V aplikácii sa používa `NavHost` a tri samostatné destinácie. Hlavná obrazovka je definovaná ako počiatočná. Obrazovka hodu a obrazovka štatistiky prijímajú identifikátor kocky ako argument v route. Vďaka tomu vedia načítať správne údaje pre vybranú položku.

```
NavHost(  
    navController = navController,  
    startDestination = "dice_selection"  
) {  
    composable("dice_selection") { ... }  
  
    composable("roll/{diceId}") { backStackEntry ->  
        val diceId = backStackEntry.arguments?.getLong("diceId")  
            ?: 0L  
        ...  
    }  
  
    composable("statistics/{diceId}") { backStackEntry ->  
        val diceId = backStackEntry.arguments?.getLong("diceId")  
            ?: 0L  
        ...  
    }  
}
```

Tento prístup je dôležitý, pretože ukazuje, že jedna aplikácia môže byť rozdelená na viac obrazoviek bez toho, aby stratila prehľadnú štruktúru. Zároveň sa tým precvičuje práca s jednoduchým argumentom v navigácii.

4 Dátový model aplikácie

Na rozdiel od CV9 tu nestačí iba jedna entita. Aplikácia pracuje s dvoma rôznymi typmi údajov. Prvým typom je samotná kocka. Druhým typom je záznam o konkrétnom hode.

Entita kocky obsahuje identifikátor, počet strán a čas vytvorenia. Entita hodu obsahuje identifikátor, odkaz na konkrétnu kocku, výsledok hodu a čas, kedy bol hod vykonaný.

```
@Entity(tableName = "dice")  
data class DiceEntity(  
    @PrimaryKey(autoGenerate = true) val id: Long = 0,  
    val sides: Int,  
    val createdAt: Long  
)
```

```

@Entity(
    tableName = "roll_entries",
    foreignKeys = [
        ForeignKey(
            entity = DiceEntity::class,
            parentColumns = ["id"],
            childColumns = ["diceId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [Index(value = ["diceId"])]
)
data class RollEntryEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val diceId: Long,
    val result: Int,
    val rolledAt: Long
)

```

Takéto rozdelenie je vhodné preto, že jedna kocka môže mať veľa hodov. Štatistika sa potom nevypočítava z jednej hodnoty, ale zo zoznamu všetkých záznamov priradených k danej kocke.

5 Room databáza a DAO vrstva

Room sa v tejto aplikácii používa na trvalé uloženie kociek aj histórie hodov. DAO rozhranie obsahuje operácie pre načítanie všetkých kociek, vyhľadanie konkrétnej kocky, načítanie všetkých hodov pre jednu kocku, vloženie novej kocky a vloženie nového hodu.

```

@Dao
interface DiceGameDao {
    @Query("SELECT * FROM dice ORDER BY sides ASC")
    fun getAllDice(): Flow<List<DiceEntity>>

    @Query("SELECT * FROM dice WHERE id = :diceId LIMIT 1")
    fun getDiceById(diceId: Long): Flow<DiceEntity?>

    @Query("SELECT * FROM roll_entries WHERE diceId = :diceId ORDER BY rolledAt DESC")
    fun getRollsForDice(diceId: Long): Flow<List<RollEntryEntity>>

    @Insert
    suspend fun insertDice(dice: DiceEntity): Long

    @Insert
    suspend fun insertRoll(roll: RollEntryEntity)
}

```

6 Repozitár a ViewModel

Rovnako ako v CV9 je aj tu medzi databázou a `ViewModel` triedou vložený jednoduchý repozitár. Jeho úlohou je sprostredkovať prácu s DAO vrstvou a oddeliť databázové operácie od logiky obrazoviek.

```
class DiceGameRepository(  
    private val diceGameDao: DiceGameDao  
) {  
    fun getAllDice(): Flow<List<DiceEntity>> = diceGameDao.  
        getAllDice()  
    fun getDiceById(diceId: Long): Flow<DiceEntity?> =  
        diceGameDao.getDiceById(diceId)  
    fun getRollsForDice(diceId: Long): Flow<List<RollEntryEntity  
        >> = diceGameDao.getRollsForDice(diceId)  
}
```

`ViewModel` trieda potom spracúva logiku celej aplikácie. Na hlavnej obrazovke drží stav vstupu pre novú kocku a zoznam dostupných kociek. Zároveň poskytuje metódy na pridanie kocky, vykonanie hodu a získanie štatistík pre konkrétnu kocku.

```
class DiceGameViewModel(application: Application) :  
    AndroidViewModel(application) {  
  
    private val _selectionUiState = MutableStateFlow(  
        DiceSelectionUiState())  
    val selectionUiState = _selectionUiState.asStateFlow()  
  
    fun onNewDiceInputChange(newValue: String) { ... }  
    fun addDice() { ... }  
    fun rollDice(diceId: Long) { ... }  
    fun getDiceStream(diceId: Long): Flow<DiceEntity?> { ... }  
    fun getStatisticsStream(diceId: Long): Flow<DiceStatistics> {  
        ... }  
}
```

Všimnite si, že v tomto cvičení sa už nepoužíva iba jeden objekt pre celú aplikáciu. `ViewModel` poskytuje aj dátové toky pre konkrétnu kocku a pre jej štatistiky. Prepájame tu logiku navigácie, databázových dát a zobrazenia detailných informácií.

7 Predvolená kocka a pridávanie vlastných kociek

Po prvom spustení aplikácie sa do databázy automaticky pridá kocka D6. Používateľ tak nemusí začínať s prázdnu aplikáciu. Neskôr si môže pridať aj vlastné kocky podľa počtu strán.

Pri pridávaní sa vstup kontroluje. Musí ísť o celé číslo, ktoré je aspoň 2. Zároveň sa kontroluje, či už takáto kocka v databáze neexistuje. Ak by sa rovnaká hodnota pridala viackrát, používateľ by mal v zozname duplicitné kocky, čo by bolo zbytočné.

```

fun addDice() {
    val input = selectionUiState.value.newDiceInput.trim()
    val sides = input.toIntOrNull()

    if (input.isBlank()) {
        ...
        return
    }

    if (sides == null || sides < 2) {
        ...
        return
    }

    viewModelScope.launch {
        val existingDice = repository.getDiceBySides(sides)
        if (existingDice != null) {
            ...
            return@launch
        }

        repository.insertDice(sides)
    }
}

```

8 Hod kockou a výpočet výsledku

Na obrazovke hodu sa po stlačení tlačidla vygeneruje náhodné číslo od 1 po počet strán danej kocky. Výsledok sa okamžite uloží do databázy ako nový záznam. To znamená, že aplikácia si neukladá iba posledný výsledok, ale kompletnú históriu hodov.

```

fun rollDice(diceId: Long) {
    viewModelScope.launch {
        val dice = repository.getDiceByIdSnapshot(diceId) ?:
            return@launch
        val result = Random.nextInt(1, dice.sides + 1)
        repository.insertRoll(diceId, result)
    }
}

```

Táto časť je zaujímavá aj tým, že spája jednoduchý výpočet s trvalým uložením údajov. Používateľ teda nevidí iba momentálny výsledok, ale buduje si históriu, z ktorej je neskôr možné vypočítat štatistiky.

9 Výpočet štatistík

Štatistika sa v tomto cvičení neukladá do databázy ako samostatná tabuľka. Namiesto toho sa vypočíta zo zoznamu hodov, ktoré už v databáze existujú. To je dôležitý návrhový princíp. Aplikácia si ukladá základné údaje, z ktorých vie neskôr odvodiť ďalšie informácie.

V `ViewModel` triede sa preto zo zoznamu hodov vypočíta počet hodov, posledný výsledok, maximum, minimum a priemer.

```
fun getStatisticsStream(diceId: Long): Flow<DiceStatistics> {
    return repository.getRollsForDice(diceId).map { rolls ->
        if (rolls.isEmpty()) {
            DiceStatistics()
        } else {
            DiceStatistics(
                totalRolls = rolls.size,
                lastResult = rolls.first().result,
                highestResult = rolls.maxOf { it.result },
                lowestResult = rolls.minOf { it.result },
                averageResult = rolls.map { it.result }.average()
            )
        }
    }
}
```

Takýto postup je vhodný, pretože uvidíte rozdiel medzi údajmi uloženými v databáze a údajmi, ktoré sa odvodzujú z už uložených záznamov.

10 Lifecycle-aware zber stavu v Compose

Dôležitou novinkou tohto cvičenia je používanie `collectAsStateWithLifecycle()`. V predchádzajúcich cvičeniach ste často čítali stav priamo z objektu alebo z jednoduchého Compose state. Tu už obrazovky získavajú dáta cez `Flow`, a preto je vhodné ich zbierať spôsobom, ktorý rešpektuje lifecycle obrazovky.

```
val diceFlow = remember(diceId) { diceGameViewModel.getDiceStream(diceId) }
val dice by diceFlow.collectAsStateWithLifecycle(initialValue = null)
```

Funkcia `collectAsStateWithLifecycle()` slúži na to, aby sa tento prúd hodnôt bezpečne premenil na Compose state. Compose potom môže s hodnotou pracovať rovnako ako s iným stavom obrazovky. Rozdiel je v tom, že zber údajov rešpektuje lifecycle obrazovky. To znamená, že údaje sa zbierajú hlavne vtedy, keď je obrazovka aktívna a viditeľná pre používateľa.

Ak by bola aplikácia presunutá do pozadia alebo by obrazovka už nebola v stave vhodnom na zobrazovanie, zber hodnôt sa nemusí zbytočne vykonávať. Aplikácia tak nemusí priebežne aktualizovať používateľské rozhranie v momente, keď ho používateľ nevidí. Keď

sa používateľ k obrazovke vráti, stav sa znovu začne zbierať a používateľské rozhranie sa aktualizuje podľa aktuálnych údajov.

V tomto cvičení je tento princíp dôležitý najmä preto, že pracujete s údajmi, ktoré sa môžu meniť mimo jednej konkrétnej composable funkcie. Hody kockou sa ukladajú do databázy, štatistiky sa z nich vypočítavajú a obrazovky potrebujú reagovať na aktuálny stav dát. `Flow` umožňuje tieto zmeny poslať smerom k obrazovke a `collectAsStateWithLifecycle()` zabezpečuje, že sa tieto hodnoty zbierajú spôsobom vhodným pre Compose aplikáciu. To znamená, že Compose obrazovka zbiera dáta iba vtedy, keď je v stave vhodnom na pozorovanie.

11 Compose obrazovky

Hlavná obrazovka používa formulár na pridanie kocky a `LazyColumn` na zobrazenie zoznamu všetkých kociek. Každá položka obsahuje názov kocky a dve tlačidlá. Jedno otvorí obrazovku hodu a druhé obrazovku štatistiky.

Obrazovka hodu pracuje iba s jednou kockou. Zobrazuje jej názov, posledný výsledok a tlačidlo na nový hod. Obrazovka štatistiky zobrazuje súhrn hodov a zoznam posledných záznamov. Všetky tri obrazovky sú vytvorené v Compose a používajú Material 3 komponenty.

12 Pridanie závislostí a KSP konfigurácie

Keďže projekt využíva Room aj Navigation Compose, do `build.gradle.kts` je potrebné doplniť príslušné závislosti. Rovnako ako v CV9 je aj tu potrebné použiť KSP konfiguráciu, pretože Room generuje časť implementácie počas buildu.

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
}

dependencies {
    implementation("androidx.room:room-runtime:2.8.4")
    implementation("androidx.room:room-ktx:2.8.4")
    ksp("androidx.room:room-compiler:2.8.4")

    implementation("androidx.navigation:navigation-compose:2.9.0")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.6")
}
```

Ak projekt používa version catalogs, je potrebné mať v `libs.versions.toml` definovaný aj plugin alias pre KSP.

13 Zhrnutie

V tomto cvičení sa po prvýkrát prirodzene spájajú Compose obrazovky, navigácia, `ViewModel`, Room databáza a lifecycle-aware zber dát. Aplikácia pritom ostáva dostatočne malá na to, aby bola vhodná aj ako cvičenie.

Na príklade vlastných kociek, hodov a štatistík si precvičíte prácu s viacerými obrazovkami, navigačný argument, uchovanie údajov v databáze, výpočet odvodených hodnôt a zobrazenie aktuálnych dát v Compose. Výsledkom bude samostatná miniaplikácia, ktorá tvorí prirodzené zakončenie celej série cvičení.